

香港中文大學
The Chinese University of Hong Kong

版權所有 不得翻印
Copyright Reserved

Course Examination 1st Term, 2024 – 2025

Course Code & Title : ENGG1110A-D Problem Solving by Programming

Time allowed : 2 hours 0 minutes Section :

Student I.D. No. : Seat No. :

請勿攜去
Not to be taken away

<< The Cover Page >>

Please read the following instructions carefully.

- You are required to answer **ALL** questions. **Full Score is 100.**
- Please write all your answers in **the space provided** on this question paper.
- **If there is not enough space, you may write answers on over-leaf.**
- Please write neatly using pen or pencil.
- Cross-out unwanted writings, otherwise, those may be considered as part of your answers.
- A list of C operators and an ASCII table are provided in the appendices.
- Unless a *complete* C program is demanded, assume proper inclusion of header files.
- For all problems, you may assume **int** is 4 bytes.
- For all problems, you may assume all input values (from keyboard or from file) are valid.
- Unless otherwise specified, all input/output are console input/output (i.e., keyboard input and screen output).

Marker's Use Only

Problem	1	2	3	4	5	6	Total
Full Score	28	12	12	13	18	17	100
Score							

Problem 1: Short Questions [28%]

Write clearly the output produced by each of the following segments of code.

Problem 1		Answers
a. 2%	<pre>int x = 5.99; printf("%d\n", (int) 0.5 + x); printf("%.2f\n", (double) x + 0.1);</pre>	
b. 3%	<pre>int i=0; for(i=1; i<3; i++){ int i = 1; i+=2; printf("%d\n", i); } printf("%d\n", i);</pre>	
c. 2%	<pre>char a[] = "a\\n"; printf("%s", a); printf("%c\n", a[2]);</pre>	
d. 2%	<pre>int a = 3; char c[] = {65, 66, 0, 67}; int i=0; while(c[i] i < a) i++; printf("char:%c\n", c[i] + 33);</pre>	
e. 2%	<pre>int a = 5, b = 4, c = b -= -a; printf("%d\n", a a); printf("%d\n", b - a);</pre>	
f. 2%	<pre>int a[4] = {3, 1, 2}; if (a[2] < a[1]); { if (a[3] < a[2]) printf("%d\n", a[0]); printf("%d\n", a[1]);</pre>	
g. 3%	<pre>void trycmp(int a[], int b[], int n){ int i = 0; for(i = 0; i < n; i++){ while (a[i] > b[i]) {a[i] -= b[i];} n--; } } int main() { int a[] = {4, 2, 9}, b[] = {2, 4, 6}; int n = 3; trycmp(a,b,n); printf("%d\n%d\n%d", a[0], a[1], n); return 0; }</pre>	
h. 3%	<pre>char a = 'a'; char s[] = "\\aAa//"; int i; for (i=0; i<strlen(s); i++) if (s[i] % a % 2) printf("%c\n", s[i]);</pre>	

Problem 1 cont'd.	Answers
i. 2% <pre>int *a; int b = 2; a = &b; int c = b*==*a; int d = b* *a; printf("%d\n", c); printf("%d\n", d);</pre>	
j. 2% <pre>struct Point { int x; int y; }; struct Point3D { struct Point a; int z; }; struct Point3D foo(struct Point a, struct Point b) { struct Point3D c; c.a = b; c.z = a.x + b.y; return c; } int main(void) { struct Point a = {1}; struct Point b = {0, 1}; struct Point3D c = foo(a,b); printf("%d\n", c.z); printf("%d\n", c.a.y); }</pre>	
k. 3% <pre>void recur(int N) { if (N == 0) return; recur(N/2); for(int i=0; i<N/2; i++) printf("*"); printf("-\n"); } void main() { recur(7); }</pre>	
l. 2% Select ONE of the three choices below that best describes elements that constitute a bribery offence. A) Accept \$100 red packet/envelop in Chinese New Year without employer's approval B) Accept a typical business meal arranged by another company C) All of the above	

Problem 2: Branching and Looping [12%]

Complete the function `fillArray` that takes a (**positive**) integer array and the `size` as input, and “interpolates” the missing entries (indicated by `-1`) from the start of the array to the end. The input array is assumed to have a **non-missing** (positive) **value** for the **first and last elements**.

The pseudocode for filling in the “-1” entries in `A` (`fillArray` function) is as follows:

Start from index `i=0` to the last array index, i.e. tranverse the array in one single pass:

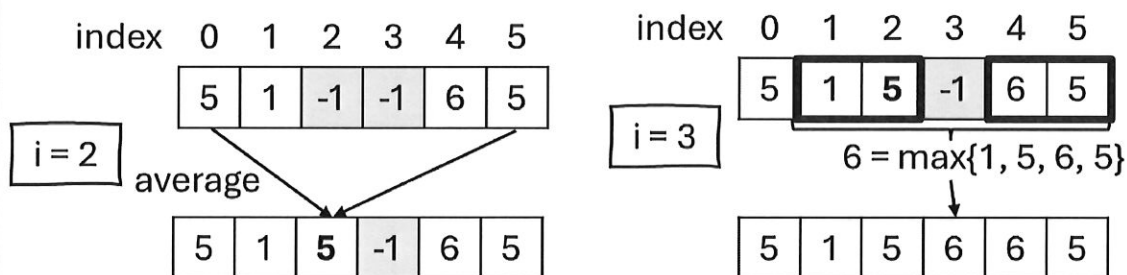
If `A[i]` is a missing entry:

- if index `i` is even: set `A[i]` to the **int** average of the first and last element.
- if index `i` is odd: set `A[i]` to the maximum of the **2 integers** on its **left** and **2 integers** on its **right**, that are **not out of bound** (total at most **4 integers**)

Print `A[i]`

Note: Interpolation uses the updated array in previous iterations.

For example, the array `A[] = {5, 1, -1, -1, 6, 5}` of size 6 has missing entries in indices 2, 3.



When $i=2$, `A[2]` updates to **5** as the result of integer division of $(A[0] + A[5])$ by 2.

When $i=3$, where `A[3] = -1`, the program uses the updated array for interpolation:

$$A[3] = \max\{A[1], A[2], A[4], A[5]\} = \max\{1, 5, 6, 5\} = 6.$$

The function should also print each entry while it is scanning through the array.

e.g., it prints 5 1 5 6 6 5

Sample runs (hardcoded program for the sample runs will result in mark deduction):

<code>A[]</code>	size	Prints
<code>{5, 1, -1, -1, 6, 5}</code>	6	5 1 5 6 6 5
<code>{1, 2, 3, -1, 5, 6, 7}</code>	7	1 2 3 6 5 6 7
<code>{2, -1, -1, -1, 3}</code>	5	2 2 2 3 3

In this **last sample run**, the array updates for $i = 1, 2, 3$:

- $i = 1$ (odd): $A[1] = \max\{A[0], A[2], A[3]\} = 2$
 - ♦ $A[-1]$ on the left is out of bound
- $i = 2$ (even): $A[2] = (2+3) / 2 = 2$
- $i = 3$ (odd): $A[3] = \max\{A[1], A[2], A[4]\} = \max\{2, 2, 3\} = 3$
 - ♦ $A[5]$ on the right is out of bound
 - ♦ $A[1], A[2]$ are the newly interpolated values

Complete the given skeleton code on the next page. You may assume necessary header files are included.

Problem 2 cont'd.

```
void fillArray(int A[], int size) {
    // scanning the whole matrix
    int i, j, max = 0;

    // loop the array with variable i
    for ( _____ ) {
        //process A[i]

        if( _____ ) { // A[i] is a missing entry

            if( _____ ) { // i is even

                A[i] =

            } else { // i is odd, you may declare other variables.
// Find max of {A[i-2], A[i-1], A[i+1], A[i+2]} & assign to A[i]

            }

        }

        // insert printf statement in the correct places below

    }

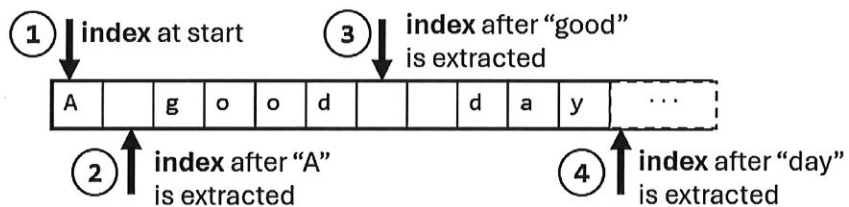
}
```

Problem 3: Character and String Processing [12%]

Implement a **string splitter** for dividing a NULL-terminated **string into words** separated by a given **separator character** (`sep_char`). Complete the function: `split()`. Before that, please read the `main()` function below to learn how to use string splitter.

```
int main( void ) {
    char str[100] = "A good  day";    // 2 spaces between "good" and "day"
    char sep_char = ' ';              // single space as the separator char
    char word[50] ;                   // for holding the extracted word string
    int  next_index , index = 0 ;     // output/input index for split()

    // split() extracts a word starting from position index in str until sep_char
    // is reached, and returns the index for the next round, or
    // -1 if no more word left
    while ( ( next_index = split( str, index, sep_char, word ) ) > 0 ) {
        printf( "%s:%d\n" , word, next_index );
        index = next_index ;
    }
    return 0 ;
}
```



With `split()` implemented, the above program should print out the followings:

```
A:1
good:6
day:11
```

Complete function `split()`, which takes the original string `str[]`, the starting position `index`, separator character `sep_char` and the to-be-extracted string `word[]`, and then it returns -1 if no word can be extracted, or returns the position in `str` just after the extracted word.

```
int split( char str[] , int index , char sep_char , char word[] )
{
    char ch ;    // for storing a single extracted character

    // Step 1 - Early termination or ignore leading sep_char: (4%)
    // iterate over the input string "str" and update position
    // variable "index", until str[ index ] is not a sep_char or ...
    while ( 1 ) {
        ch = str[ index ] ;    // get the next char from index position
        // Please fill your code
    }
}
```

Problem 3 cont' d.

```
        index++ ;        // update index for getting the next char
    }

    // Step 2 - Word Extraction:
    // copy contents of string "str" to "word" until "str[ index ]" is a
    // sep_char or ... (8%)

    int idx_word = 0;    // an index for accessing word's char
    ch = str[ index ] ; // get the first char of the word

    while ( _____ ) {

        // ... (8%) ...

    } // end while
    // extra terminating steps here

    return index ; // returns index in str just after the extracted word
} // end of function
```

Problem 4: Array Processing [13%]

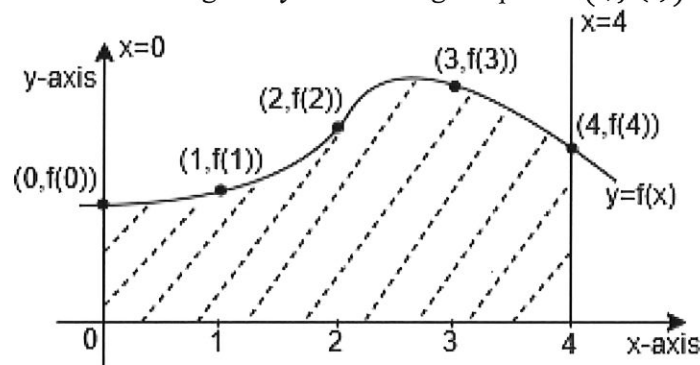
Simpson's rule is a tool to approximate the area bounded by a curve $y = f(x)$, the x-axis, and the vertical lines $x = 0$ and $x = N$ for some even $N \geq 2$; assuming $f(x)$ is non-negative in the given range of x . Given the coordinates $(i, f(i))$ for $i = 0, 1, \dots, N$, Simpson's rule states that the desired area can be approximated by the following formula:

$$\sum_{j=0}^{\frac{N}{2}-1} \frac{1}{3} [f(2j) + 4f(2j+1) + f(2j+2)]. \quad \dots\dots\dots(1)$$

For example, if $N = 4$, the approximation is:

$$\begin{aligned} & \frac{1}{3} [(f(0) + 4f(1) + f(2)) + (f(2) + 4f(3) + f(4))] \\ &= \frac{1}{3} [f(0) + 4f(1) + 2f(2) + 4f(3) + f(4)]. \end{aligned}$$

This can be illustrated by the following figure. The curve on top is $y = f(x)$, the y-axis is the vertical line $x = 0$, and the vertical line on the right side is $x = N = 4$. Simpson's rule approximates the shaded area in the figure by considering the points $(i, f(i))$ for $i = 0, 1, 2, 3, 4$.



- a) Complete the following function that returns the approximated area bounded by $f(x)$ and the x-axis for $0 \leq x \leq N$ by Simpson's rule, i.e., Equation (1) above. (4%)

```
// The evaluation of f(x), in double, is stored in f[x]
// e.g., f(0) is stored in f[0], f(1) is stored in f[1], etc.
// N below is that in the Simpson's rule formula
// Beware that the number of points is not exactly N
```

```
double simpson(double f[], int N) {
```

```
}
```


Problem 4 cont'd.

- b) Consider a triangle formed by the points (0,0), (1,1) and (2,0). Complete the following code fragment that approximates the area of this triangle by calling the function in part (a). (3%)

```
double area; // for storing the answer
```

```
printf("%f\n", area);
```

- c) Consider the curve $f(x) = \frac{x^3}{2}$ and its evaluations at $x = 0, 1, \dots, 50$. Complete the following code fragment that approximates the area bounded by $f(x)$, the x-axis, and the vertical lines $x = 0$ and $x = 50$ by calling the function in part (a). (6%)

```
double area; // for storing the answer
```

```
printf("%f\n", area);
```

Problem 5: 2D Array and File Processing [18%]

Please complete Part A and Part B of the following C program (reading **tic-tac-toe** game boards).

```
#include <stdio.h>
#include <stdlib.h>

int checkRowCol(int A[][3]){
// TODO: Complete Part A(a)
}

int checkDiag(int A[][3]){
// TODO: Complete Part A(b)
}

/* TODO: Complete Part B */
/* End of Part B */
```

**Example file
input.txt**

3

0 1 2

1 1 2

0 0 2

1 2 1

2 1 1

2 2 1

1 1 2

2 2 1

1 1 2

	O	X
O	O	X
		X

Player 2 (X) wins

O	X	O
X	O	O
X	X	O

Player 1 (O) wins

O	O	X
X	X	O
O	O	X

Draw

Sample Output

2

1

0

In the game board, **0** represents **empty** cell; **1** represents player **1**'s move (**O**) and **2** for player **2** (**X**).

The program utilizes 2 functions `checkRowCol` and `checkDiag` to check if there are matches (3 of the same player's move in any row, column or diagonal) in the game board.

Part A: 2D Array Processing (7%)

In this part, you may assume a 3x3 *finished* game board (3x3 2D integer array `int A[3][3]`) is supplied as an argument to the functions. The functions will return the winning player (as integer 1 or 2) for satisfying the **row / column / diagonal** winning condition; and return 0 otherwise.

For example, for `int A[3][3]={{0,1,2},{1,1,2},{0,0,2}}` (the first game in the example file above), `checkDiag(A)` returns 0 but `checkRowCol(A)` return 2.

All game boards are assumed valid (either draw or one of the players wins), and no validation is required. Assume necessary headers are included.

- a) Complete the function `checkRowCol` to return the winner (1 or 2) if all elements (player's moves) in any row or any column are the same in the 3x3 2D array A, return 0 otherwise. (5%)

```
int checkRowCol (int A[][3]){
```

Problem 5 cont'd.

```
}
```

- b) Complete the function `checkDiag` to return the winner (1 or 2) if elements in either one of the diagonals are the same, 0 otherwise. **(2%)**

```
int checkDiag(int A[][3]){
```

```
}
```

Problem 5 cont'd.

Part B: File IO and 2D Array (11%)

The program reads finished tic-tac-toe game boards from `input.txt`, and writes the winners to the console. The first number indicates the number of game boards, denoted as N , where $1 \leq N \leq 100$. `input.txt` contains **at most 100** game boards. You should not declare other arrays unless you are told to do so.

If the file cannot be opened successfully, *the program will terminate immediately*. However, if the file can be opened, you can assume that the file format is correct, and no validation is required.

Complete the skeleton code to be placed to the main program to

- 1) Process N game boards stored in the file "input.txt".

Note: You can only use ONE 3x3 2D array to process the game board one by one.

- 2) Using the functions in Part A to determine and output the winner (1 or 2) or 0 (for draw).

**Example file
`input.txt`**

```
3
0 1 2
1 1 2
0 0 2
1 2 1
2 1 1
2 2 1
1 1 2
2 2 1
1 1 2
```

```
int main(){
    FILE *in;
    int i, j, N; // useful variables

    // Declare the 3x3 array A for storing the game board


    // Open the file "input.txt"


    // Exit if the file "input.txt" fails to open


    // Read the first integer from the file "input.txt" to N
```

Problem 5 cont'd.

```
// Read N game boards one-by-one,  
// Store each board to 2D array A, and  
// Print each winner.  
// You may declare (non-array) helper variables.
```

```
return 0;
```

```
}
```

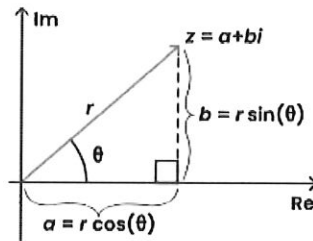
Problem 6: Struct and Functions [17%]

Every complex number z can be represented in standard form $z = a + bi$ where a and b are real numbers, and $i = \sqrt{-1}$. z is a real number if and only if $b = 0$. On the other hand, every $z \neq 0$ can be represented in polar form $z = r\angle\theta$ where $r > 0$ and the angle notation ($\angle$) denotes θ as an angle in the range: $-\pi < \theta \leq \pi$.

Given the following C structures for representing a complex number, where struct `standard` is for the standard form and struct `polar` is for the polar form.

```
struct standard {
    double a, b; // a+bi
};

struct polar {
    double r, ang; // r∠ang
};
```



- a) Write a code segment to alias struct `standard` as `c_std` and struct `polar` as `c_pol`. (2%)

- b) Given r and θ (in radians), we can compute the standard form parameters by $a = r \cos \theta$ and $b = r \sin \theta$. On the other hand, given a and b , we can compute the polar form parameters by $r = \sqrt{a^2 + b^2}$ and $\theta = \arctan(\frac{b}{a})$. You shall find the following built-in math functions in C useful, where their function prototypes are given below:

```
double cos(double theta); // takes one argument: theta in radians, and
                          // returns the calculated value of cosine(theta)

double sin(double theta); // takes one argument: theta in radians, and
                          // returns the calculated value of sine(theta)

double atan2(double b, double a); // takes two arguments: b and a, and
                                // returns the calculated angle, arctangent of b/a, in radians
```

Complete the following functions that convert a complex number from standard form to polar form, as well as from polar form to standard form. (5%)

```
c_pol to_pol(c_std z) { // assume z is non-zero
```

```
}
```

Problem 6 cont'd.

```
c_std to_std(c_pol z) {
```

```
}
```

c) Suppose the **addition of complex numbers has been implemented**, and given as the following function prototype:

```
c_std add(c_std z_std1, c_std z_std2); // z_std1 + z_std2
```

Complete the following code segment that fills in the array `z` with complex numbers $2x + xi$ for $x = 1, 2, \dots, 50$, and stores the sum of these numbers in the variable `sum`. The sum must be computed by calling the function `add()` given above. (4%)

```
c_std z[50]; // fill the complex numbers in this array
c_std sum; // store the answer in this variable
```

```
// end of 6(c)
```

Problem 6 cont'd.

d) Suppose the substruction and multiplication of complex numbers **have been implemented**, and given as the following function prototypes:

```
c_std sub(c_std z_std1, c_std z_std2); // z_std1 - z_std2
c_pol mul(c_pol z_pol1, c_pol z_pol2); // z_pol1 * z_pol2
```

Write a function called `discriminant` that takes three complex numbers a, b, c in polar form as input, and then returns the value of $b^2 - 4ac$ **in standard form** by calling the arithmetic functions given above and the conversion functions in part (b). Beware the data types in the prototypes of the arithmetic functions. (6%)

Hints: addition and substruction are easier to perform in standard form, while multiplication and division are easier to perform in polar form.

// end of 6(d)

<< Appendix 1 >>

Partial List of C Operators in Decreasing Precedence						Associativity
()	[]	.	->	++ (postfix)	--	left-to-right
			(postfix)			
+	(unary)	-	(unary)	++ (prefix)	--	right-to-left
			(prefix)			
		!		* (unary)		
		*	/	%		left-to-right
+	(addition)	-	(subtraction)			left-to-right
		<	<=	>	>=	left-to-right
			==	!=		left-to-right
			&			left-to-right
			&&			left-to-right
						left-to-right
=	+=	-=	*=	/=	etc.	right-to-left
		,	(comma operator)			left-to-right

<< Appendix 2 >>

ASCII Table							
0 NUL	1 SOH	2 STX	3 ETX	4 EOT	5 ENQ	6 ACK	7 BEL
8 BS	9 HT	10 NL	11 VT	12 NP	13 CR	14 SO	15 SI
16 DLE	17 DC1	18 DC2	19 DC3	20 DC4	21 NAK	22 SYN	23 ETB
24 CAN	25 EM	26 SUB	27 ESC	28 FS	29 GS	30 RS	31 US
32 SP	33 !	34 "	35 #	36 \$	37 %	38 &	39 '
40 (	41)	42 *	43 +	44 ,	45 -	46 .	47 /
48 0	49 1	50 2	51 3	52 4	53 5	54 6	55 7
56 8	57 9	58 :	59 ;	60 <	61 =	62 >	63 ?
64 @	65 A	66 B	67 C	68 D	69 E	70 F	71 G
72 H	73 I	74 J	75 K	76 L	77 M	78 N	79 O
80 P	81 Q	82 R	83 S	84 T	85 U	86 V	87 W
88 X	89 Y	90 Z	91 [	92 \	93]	94 ^	95 _
96 `	97 a	98 b	99 c	100 d	101 e	102 f	103 g
104 h	105 i	106 j	107 k	108 l	109 m	110 n	111 o
112 p	113 q	114 r	115 s	116 t	117 u	118 v	119 w
120 x	121 y	122 z	123 {	124	125 }	126 ~	127 DEL

<< END OF PAPER >>